

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 620 — Computational Geometry

Hulls, triangulations, subdivision, and proximity — with the robustness problem treated as the subject rather than an appendix

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 5 (with CSCE 605 and CSCE 678)

Prerequisites: CSCE 629 Analysis of Algorithms; CSCE 641 Computer Graphics; CSCE 645 Geometric Modeling is helpful but not required

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A geometry library whose predicates are exact, whose degeneracies are handled by name, and whose test suite is built from the cases that break naive implementations

Course description

Computational geometry is the one subject in this program where **a correct algorithm, correctly implemented, routinely produces a wrong answer**. Not slowly, not approximately — wrong, and often catastrophically: a convex hull that is not convex, a triangulation with a hole in it, a point-in-polygon test that says both yes and no depending on which edge you ask.

The reason is that **geometric algorithms are built on predicates** — questions like *is this point left of that line?* — and the textbook proofs assume those predicates are answered exactly. Floating point does not answer them exactly. It answers them *nearly* exactly, which for a combinatorial algorithm is worse than useless: a single inconsistent answer can drive the algorithm into a state its correctness proof never contemplated, and from there it may loop forever, crash, or quietly emit nonsense.

So this course teaches the algorithms and the robustness problem together, which is how practitioners actually have to hold them. **Module 02 builds exact predicates before any algorithm needs them**, and every module afterwards says what its degeneracies are and how they are handled.

The emphasis throughout is on the geometry an engine needs. **Convex hulls and decomposition feed collision detection; triangulation and Delaunay feed meshing; BSP trees, quadtrees, and k-d trees feed culling and spatial queries; visibility and configuration space feed navigation**. CSCE 649 assumed collision geometry worked. This course is where it comes from.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Explain why floating-point predicates break combinatorial algorithms, and quantify when.
- 2 Implement exact and adaptive-precision geometric predicates.
- 3 Implement convex hulls in two and three dimensions.
- 4 Apply the plane-sweep paradigm to intersection and related problems.
- 5 Triangulate polygons and perform boolean operations on them.
- 6 Build and query BSP trees, k-d trees, and quadtrees or octrees.
- 7 Construct Voronoi diagrams and Delaunay triangulations and explain their duality.
- 8 Answer range, proximity, and nearest-neighbour queries efficiently.
- 9 Compute visibility and plan motion in configuration space.
- 10 Choose between writing geometry and using a library, with reasons.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	Geometry a Machine Can Believe	Why do correct geometric algorithms produce wrong answers?
2	Exact and Adaptive Predicates	How do you answer a predicate exactly, and quickly?
3	Convex Hulls	What is the smallest convex set containing these points?
4	Plane Sweep and Segment Intersection	How do you avoid comparing everything with everything?
5	Polygon Triangulation	How do you cut a polygon into triangles?
6	Polygon Boolean Operations	How do you combine two shapes?
7	Spatial Subdivision	How do you organise space so queries are fast?
8	Voronoi Diagrams	Which site is closest to each point of the plane?
9	Delaunay Triangulation and Meshing	Of all triangulations of these points, which is best?
10	Range Searching and Proximity	What is near this point, and what is in this box?
11	Visibility and Motion Planning	What is visible from here, and how do I move without colliding?
12	Geometry in Three Dimensions	How do you answer geometric queries on real 3D assets?
13	Shipping Geometric Code	How do you make geometric code that survives real input?

Projects

Project 1 — A robust 2D geometry kernel (after Module 7)

Build the foundation layer: exact predicates, convex hull, segment intersection, triangulation, and boolean operations — each one tested against the degeneracies that break it.

Requirements

- **Exact orientation and incircle predicates**, with an adaptive fast path, tested against exact rational arithmetic on inputs chosen to be near-degenerate.
- A convex hull that is correct on **collinear points, duplicate points, and all-identical points**, not merely on random clouds.
- Bentley–Ottmann segment intersection handling **coincident endpoints and three-or-more segments through one point**.
- Polygon triangulation that handles **holes**.
- **A degenerate-input test suite**, written before the code it tests.
- SVG output for every structure, used in the report.

Done means

- **A test suite in which every test is a case that breaks a naive implementation**, with a note on each saying what it breaks and why.
- A demonstration, with figures, that your hull is correct on input where a double-based orientation test produces a non-convex result.
- **A measured comparison of the adaptive predicate against both the naive and the fully exact versions** — showing the fast path is actually fast.
- **An honest list of the degeneracies you did not handle**, which is a legitimate engineering outcome when stated.

Project 2 — Spatial structure, applied (after Module 12)

Take the kernel into three dimensions and into a query-answering structure. One structure, built properly, measured against the alternatives.

Requirements

- **Either** a Delaunay triangulation with a constrained variant and a quality-meshing pass; **or** a 3D spatial structure (BVH, k-d tree, or octree) with range and nearest-neighbour queries; **or** a 3D convex hull with GJK collision queries on top.
- Construction and query cost both measured, over varying input size and distribution.
- **A comparison against CGAL or an equivalent library** on correctness and on speed.

- **A clustered or adversarial input distribution**, not only uniform random — uniform random hides almost every interesting failure.
- Degeneracy handling stated explicitly.

Done means

- **Construction and query timings against input size and distribution**, plotted, with the asymptotic prediction overlaid and any divergence explained.
- **An honest comparison against the library version**, covering both correctness and performance.
- A statement of which input distributions degrade your structure and by how much.
- **A rendered figure of the structure on real input**, because geometry that cannot be looked at has not been checked.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
David Mount — CMSC 754 Computational Geometry lecture notes (free PDF)	The primary source. A complete graduate course in one carefully written document, free, and better paced than most textbooks. Read alongside every module.
de Berg, Cheong, van Kreveld & Overmars — Computational Geometry: Algorithms and Applications	The standard text, and the source of the canonical treatments of sweep, Voronoi, and range searching. Library copy; the authors' course slides are free.

Resource	Role
Jonathan Shewchuk — Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates (free)	The reference for Module 02 , with working code. The paper that made exact predicates practical rather than theoretical.
CGAL — the Computational Geometry Algorithms Library	The reference implementation of nearly everything in this course. Its documentation is itself a good secondary source, and its kernel design is the subject of Module 13.
Christer Ericson — Real-Time Collision Detection	The engine-facing companion. Where this course gives the algorithm, Ericson gives the shipping version. Library copy; the author's errata and notes are free.
TU Eindhoven — Computational Geometry course materials (free)	Slides and exercises from one of the groups that wrote the standard text.

Tooling

- **C++ or Rust**, because Module 02 needs control over floating-point evaluation and both give it to you. **A language that silently reassociates your arithmetic cannot do this course.**
- **Shewchuk's predicates.c**, or your own implementation of it. Module 02 writes one; after that you may use his.
- **A way to draw what you computed.** SVG output is enough and takes an afternoon. **Geometric bugs are visible and almost nothing else about them is** — debugging a triangulation from printed coordinates is a waste of a week.
- **Exact rational arithmetic** available for testing — Python's `fractions.Fraction`, `Boost.Multiprecision`, or `GMP`. Used to produce ground truth, not to ship.
- **A degenerate-input generator.** Collinear points, duplicate points, cocircular points, coincident edges. **You will write this in Module 01 and use it in every module after.**
- **CGAL installed**, for Module 13's comparison and for checking your answers throughout.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a `TODO` naming what you do not understand, and go back to it.